

leet code

1295 Find Numbers with Even Number of Digits

I/p : num = [12, 345, 2, 6, 7896]

O/p : 2

```
→ class Solution {
    public int findNumbers(int[] nums) {
        int count = 0;
        for (int i = 0; i < nums.length; i++) {
            if (isEven(nums[i])) {
                count++;
            }
        }
        return count;
    }
    private boolean isEven(int num) {
        int count = 0;
        while (num > 0) {
            count++;
            num /= 10;
        }
        return count % 2 == 0;
    }
}
```

@ WAP to swap all elements in a given Array.

=>

```
public class SwapArray {
```

```
    main () {
```

```
        int[] arr = {5, 8, 7, 6, 2};
```

```
        System.out.println(Arrays.toString(arr));
```

```
        swap(arr);
```

```
        System.out.println(Arrays.toString(arr));
```

```
    }
```

```
    public static void swap(int[] a) {
```

```
        int i = 0; int j = a.length - 1;
```

```
        while (i < j) {
```

```
            [ int temp = a[i];
```

```
              a[i] = a[j];
```

```
              a[j] = temp;
```

```
            [ a[i] = a[i] + a[j];
```

```
              a[j] = a[i] - a[j];
```

```
              a[i] = a[i] - a[j];
```

```
              i++;
```

```
              j--;
```

```
            }
```

```
    }
```

Q. WAP to swap all character present in given Array.

a	e	i	o	u
---	---	---	---	---

 $\Rightarrow$

u	o	i	e	a
---	---	---	---	---

→

```
public class SwapCharacter {  
    public static void main(String[] args)  
    {  
        char[] ch = { 'a', 'e', 'i', 'o', 'u' };  
        System.out.println(Arrays.toString(ch));  
        swapArray(ch);  
        System.out.println(Arrays.toString(ch));  
    }
```

```
    public static void swapArray(char[] ch)  
    {
```

```
        int i = 0;  
        int j = ch.length - 1;  
        while (i < j) {
```

```
            int int temp = ch[i];  
            ch[i] = ch[j];  
            ch[j] = temp;  
        }  
    }
```

(without using
3rd variable)

```
}
```

o/p :

u o i e a.

Leetcode: 1464

maximum product of two elements in an array.

→ class Solution {

public int maxProduct(int[] nums){

int max1 = Integer.MIN_VALUE;

int max2 = Integer.MIN_VALUE;

for (int i : nums){

~~if~~ if (i > max1){

max2 = max1;

max1 = ~~max~~ i;

}

else if (i > max2)

{ max2 = i;

}

}

return (max1-1)*(max2-1);

}

Q WAP to find frequency of each element in a given Array.

```

→ public class Frequency {
    public static void main(String[] args) {
        int[] a = {1, 1, 2, 1, 1, 5, 5, 3, 1, 3};
        freq(a);
    }
    public static void freq(int[] arr) {
        int min = min(arr);
        int max = max(arr);
        int[] count = new int[max + 1];
        for (int i = min; i < arr.length; i++) {
            if (count[i] != 0) {
                count[arr[i]]++;
            }
        }
    }
}

```

```

public static int min {
    for (int i = min; i < count.length; i++) {
        {
            if (count[i] != 0) {
                System.out.println(i + " ----> " + count[i]);
            }
        }
    }
}

```

```

public static int min(int[] num) {
    int min = Integer.MAX_VALUE;

```

```

    for (int i : num) {
        if (i < min) {
            min = i;
        }
    }

```

```

    return min;
}

```

```

public static int max(int[] num) {
    int max = Integer.MIN_VALUE;
    for (int i : num) {
        if (num[i] > max) {

```

max = ~~arr~~ nums[0];

}

} return max;

}

- Missing - $\text{count}(i) == 0$
- Unique - $\text{count}(i) == 1$
- Duplicate - $\text{count}(i) > 1$

(2) Binary Search

- It will work only on sorted Array.
- Time complexity of Binary search is $O(\log n)$.
- When we are using binary search 1st we will find middle index & we will compare middle element with target element.
- if both are same we will return index of that element otherwise we will check whether element is greater than middle element. if condition satisfies we will update i value otherwise we will update j value.
- we will continue same procedure until condition become false.
- if element is not found return -1 .
- When we are using binary search it will not search entire Array from 0 to last index.
- It will search only half of the Array and it will skip half of the iteration because of this reason, time complexity of binary search is $O(\log n)$.

• Example

(a) IP: $nums = [1, 0, 3, 5, 9, 12]$, $target = 9$
OP: 4
 9 exists in $nums$ & its index is 4

```

→ class Solution {
    public int search(int[] nums, int target)
    {
        int i = 0, j = nums.length - 1;
        while (i <= j) {
            int mid = (i + j) / 2;
            if (nums[mid] == target) {
                return mid;
            }
            else if (target > nums[mid]) {
                i = mid + 1;
            }
            else {
                j = mid - 1;
            }
        }
        return -1;
    }
}

```

OR

```

→ public static int RecursionBinary(int[] nums, int target,
    int i, int j)
{
    if (i > j) { return -1; }
    int mid = (i + j) / 2;
    if (nums[mid] == target) {
        return mid;
    }
    else if (target > nums[mid]) {
        i = mid + 1;
    }
    else {
        j = mid - 1;
    }
    return Recur RecursionBinary(nums, target, i, j);
}

```

Q WAP to shift All Array elements left hand side by k th position.

i/p:

10	20	30	40	50
----	----	----	----	----

 $k=2$

o/p:

30	40	50	10	20
----	----	----	----	----

i/p:

10	20	30	40	50
----	----	----	----	----

$k=3$

o/p:

40	50	10	20	30
----	----	----	----	----

i/p:

10	20	30	40	50
----	----	----	----	----

$k=5$

o/p:

10	20	30	40	50
----	----	----	----	----

```
import java.util.Arrays;
→ public class leftShiftByKth {
    public static void main(String[] args)
    {
        int[] a = {10, 20, 30, 40, 50};
        System.out.println(Arrays.toString(a));
        leftShiftByK(a, 2);
        System.out.println(Arrays.toString(a));
    }
    public static void leftShiftByK(int[] nums, k)
    {
        int[] res = new int[nums.length];
        for (int i=0; i<nums.length; i++)
        {
            res[i] = nums[(i+k)%nums.length];
        }
        for (int i=0; i<res.length; i++)
        {
            num[i] = res[i];
        }
    }
}
```

Q) WAP to shift Rightshift element from Array By
kth position (189 LC), 4 1822 LC

→ import java.util.Arrays;

```
public class RightShiftEle {
```

```
    public static void main (String[] args) {
```

```
        int[] nums = { 10, 20, 30, 40, 50 };
```

```
        System.out.println(Arrays.toString(nums));
```

```
        rightShiftByK (nums, 2);
```

```
        System.out.println(Arrays.toString(nums));
```

```
    } public static void rightShiftByK (int[] nums, int k)
```

```
    {
```

```
        int[] res = new int[nums.length];
```

```
        for (int i=0; i<nums.length; i++) {
```

```
            nums res[i+k % nums.length] = nums[i];
```

```
        }
```

```
        for (int i=0; i<res.length; i++) {
```

```
            nums[i] = res[i];
```

```
        }
```

```
    }
```

```
}
```

122. LeetCode.

@ sign of the product of an Array.

0=0 -res = -1 +res = 1

→ public class Solution {

public static int productArray(int [] nums)

{
 int count = 0;

~~if~~

for (int i = 0; i < nums.length; i++)

{

 if (nums[i] == 0) {

 return 0;

 }

 if (nums[i] < 0) {

~~return 1;~~ count++;

 }

~~else~~

~~return 1;~~

~~}~~

}

return count % 2 == 0 ? 1 : -1;

}

}

409 Leetcode

to lower case

→ class solution {

public ~~static~~ String toLowerCase (String s)

{

char[] originalData = s.toCharArray();

char[] modifiedData = new char[originalData.length];

for (int i = 0; i < originalData.length; i++)

{
if (originalData[i] >= 'A' & originalData[i] <= 'Z')

{
modifiedData[i] = (char) (originalData[i] + 32);

}
else {

modifiedData[i] = originalData[i];

}
return new String(modifiedData);

}

344 reverse string

→ class solution {

~~int i = 0;~~ ~~String~~ void reverseString (char s)

{
int i = 0; int j = s.length - 1;

~~for~~ while (i < j) {
char temp = s[i];
s[i] = s[j];
s[j] = temp;
i++;
j--;

}

}

}

}

1844

Replace All Digits with characters

class solution {

you have given a 0-indexed string that has lower case English letters. & it's even index.

& its odd indices has digit.

ip: s = "a1c1e1"

op: "abcdef"

ip: s = "a1b2c3d4e"

op: "abbdcfdhe"

class solution {

public string replaceDigit(String s)

{ char[] data = s.toCharArray();

for (int i = 1; i < data.length; i += 2)

{ int num = data[i] - '0';

data[i] = (char) (data[i-1] + num);

} return new String(data);

2108

string

class

public

for

{

}

}

return

private

{

ce

}

{

}

}

}

}

re

}

}

2108 string palindrom i/p: ["abc", "car", "ada", "wow"]
o/p: "ada"

```
class Solution {
public String firstPalindrome(String[] words) {
    for (int i = 0; i < words.length; i++) {
        {
            if (isPalindrome(words[i])) {
                return words[i];
            }
        }
    }
    return "";
}

private boolean isPalindrome(String s) {
    {
        int i = 0, j = s.length - 1;
        while (i < j) {
            {
                if (s.charAt(i) != s.charAt(j)) {
                    {
                        return false;
                    }
                }
                i++;
                j--;
            }
        }
        return true;
    }
}
```

1832 vimp

13/09/2022

② check if the sentence is Pangram.

class Solution {

public boolean checkIfPangram(String sentence)

{
if (sentence.length() < 26)
return false;

for (char ch = 'a'; ch <= 'z'; ch++)

{
if (sentence.indexOf(ch) == -1)
{
return false;
}
}

return true;

}

}

① i/p: sentence = "the quick brown fox jumps over the lazy dog"
o/p: true

(a-z) ✓

② i/p: sentence = "leetcode"
o/p: false

(a-z) ✗

1704

Determine if string is a pangram

i/p: "book"
o/p: true

class Solution {

public boolean checkIfPangram(String sentence)

{
int[] arr = new int[26];
for (int i = 0; i < 26; i++)

{
if (sentence.indexOf(arr[i]) == -1)
return false;
}

return true;
}

}

1704

Determine if String Halves Are Alike

inp: "book"
 o/p: true

bo → 1
 ok → 1
 1 == 1 → true

→ class solution {

public boolean isHalve (String s)

{

int mid = s.length() / 2;

int firstHalf = 0;

int secondHalf = 0;

for (int i = 0; i < mid; i++) {
 if (isVowel (s.charAt(i))) {
 firstHalf++;
 }

for (int i = mid; i < s.length(); i++)

{
 if (isVowel (s.charAt(i))) {
 secondHalf++;
 }

} return firstHalf == secondHalf;

} private boolean isVowel (char ch) {

return ch == 'a' || ch == 'e' || ch == 'i' || ch == 'o'

|| ch == 'u' || ch == 'A' || ch == 'E' || ch == 'I'

|| ch == 'O' || ch == 'U';
 }

}

}

1768

Merge String Alternately

Example: ① I/p: word1 = "abc" word2 = "pqr"

O/p: apbqcr

② I/p: word1 = "ab" word2 = "pqr"

O/p: apbqcr

```
→ class Solution {  
    public String mergeAlternately(String word1,  
                                   String word2)  
    {  
        String merge = "";  
        int i = 0;  
        int j = 0;  
        while (i < word1.length() || j < word2.length())  
        {  
            if (i < word1.length())  
            {  
                merge = merge + word1.charAt(i);  
                i++;  
            }  
            if (j < word2.length())  
            {  
                merge = merge + word2.charAt(j);  
                j++;  
            }  
        }  
        return merge;  
    }  
}
```

S20 Detect Capital

we define the usage of capitals in a word to be right when one of the following cases hold

- All letters in this word are capitals, like "USA".
 - All letters in this word are not capitals, like "leetcode".
 - only the first letter in this word is capital, like "Google".
- then return true.

i/p: "USA" o/p: true

i/p: FLAG o/p: false.

```
→ class Solution {
    public boolean detectCapital(String word) {
        int upper = 0;
        for (int i = 0; i < word.length(); i++) {
            char ch = word.charAt(i);
            if (ch >= 'A' && ch <= 'Z') {
                upper++;
            }
        }
        if (upper == word.length()) {
            return true;
        }
        if (upper == 0) {
            return true;
        }
        if (upper == 1 && (word.charAt(0) >= 'A' && word.charAt(0) <= 'Z')) {
            return true;
        }
        return false;
    }
}
```

Ques

242 valid Anagram

i/p : s = "anagram", o/p : true
t = "nagaram"

i/p : s = "rat",
t = "car". o/p : false.

→ class Solution {

public boolean isValidAnagram(String s, String t)

{
 if (s.length() != t.length()) {
 return false;
 }

 int[] isPresent = new int[26];

 for (int i = 0; i < s.length(); i++)

 {
 char ch1 = s.charAt(i);
 char ch2 = t.charAt(i);
 isPresent[ch1 - 'a']++;
 isPresent[ch2 - 'a']--;

 for (int i = 0; i < isPresent.length; i++)

 {
 if (isPresent[i] != 0) {
 return false;
 }

 return true;
}

}